

Assessment 2

Group 10 Members:

**Ben Walker, Josh White, Chris Williams, Gavindu Tissera, Nana
Twum Oviarobo, Tamerlan Urazbayev, Tymur Topala**

**Software Testing
Report**

Summary

As part of software testing, we have implemented various methods and approaches to ensure quality testing. These include automated tests as implemented using the JUnit testing framework, test doubles such as stubs (implemented through the Mockito framework) and exploratory testing for more manual sections of the project that require human judgement. To keep track of the automated unit tests we use the JaCoCo (Java Code Coverage) Library. This outputs a [document](#), detailing what percentage of the code is covered by unit tests.

With these multiple approaches combined, our software testing can reach all important areas of our code and as such, allows us to find issues and bugs throughout. Although this was achieved by a variety of methods, it was appropriate to automate the tests as much as possible, as done through JUnit (found in our [testing package](#)). This benefits our project and was fitting as it resulted in improved workflow, both in terms of efficiency and between team members. Beyond the given benefits of automated testing (e.g. less debugging, better changes, better code reviews and faster releases), automation was especially fitting for a project such as this due to the large number of classes and methods. This was also very useful when it came to decomposing the testing and allowing work to be split between people.

Given that a lot of our code is creating and manipulating UI elements, we felt that it was more conducive to conduct manual testing on pull requests. These consisted of verifying that the code works as expected, visually.

Report

Concerning the tests themselves, we have a wide variety of automated tests within the game using JUnit. These cover the majority of the areas of the game (including but not limited to achievements, assets, camera, events, input and timer). All these tests need to pass in order to build the executable jar file for the game, resulting in reduced risks with bugs and meaning that problems which result in failed tests are far less likely to be forgotten about and significantly affect future dependent areas of code if left unchecked when scaling. Regarding our JaCoCo structural coverage, we also have a high level of coverage across the board, allowing us to exercise the code and catch any important problems (such as edge cases).

Components without automated tests

While we pushed to make as much of the game covered by automated tests as possible, some components - namely UI - cannot be tested through JaCoCo. Instead we focused more on manual tests for these elements to avoid having usability and functionality issues. The process of testing the UI was having an individual go through buttons and menus, and evaluate the usability and accessibility of each interaction. While this process would usually be more time consuming and labour intensive than automated testing, we found it would be more appropriate for this project for two main reasons.

Firstly, we found the game UI to be simple enough that we could test the UI to completion without an excessive amount of effort on our end, and the upfront effort to set up and maintain automated testing would be higher than the effort required for manual testing. As well as that, manual testing allowed us to evaluate the UI from a user perspective, which we would be unable to do with automated testing alone.

We also decided to apply manual testing when testing against our requirements, since while automated testing covered a large proportion of our code, a lot of these requirements needed human evaluation to assess correctly.

Automated testing

We split our test package into different classes, each focusing on a specific part of the code. This improves the maintainability of the tests, and gives us an easy way of seeing what part of the test isn't passing. As we use the headless launcher, we need to substitute in the calls the test makes to gdx, and other graphic libraries. As described above, we went with mockito, as this allowed for a greater flexibility in use given its integration with JUnit. To reduce repeated code, JUnit contains the `@BeforeEach` decorator, which gets run before each test. This ensures that each test is run from the same starting conditions.

At the time of writing, we have 84 unit tests with 100% pass rate. The test takes about 8 seconds to run, with the longest single unit test taking 5. While the tests could be slightly more thorough, we feel like there is enough automated coverage that other edge cases will be picked up in manual testing and code review.